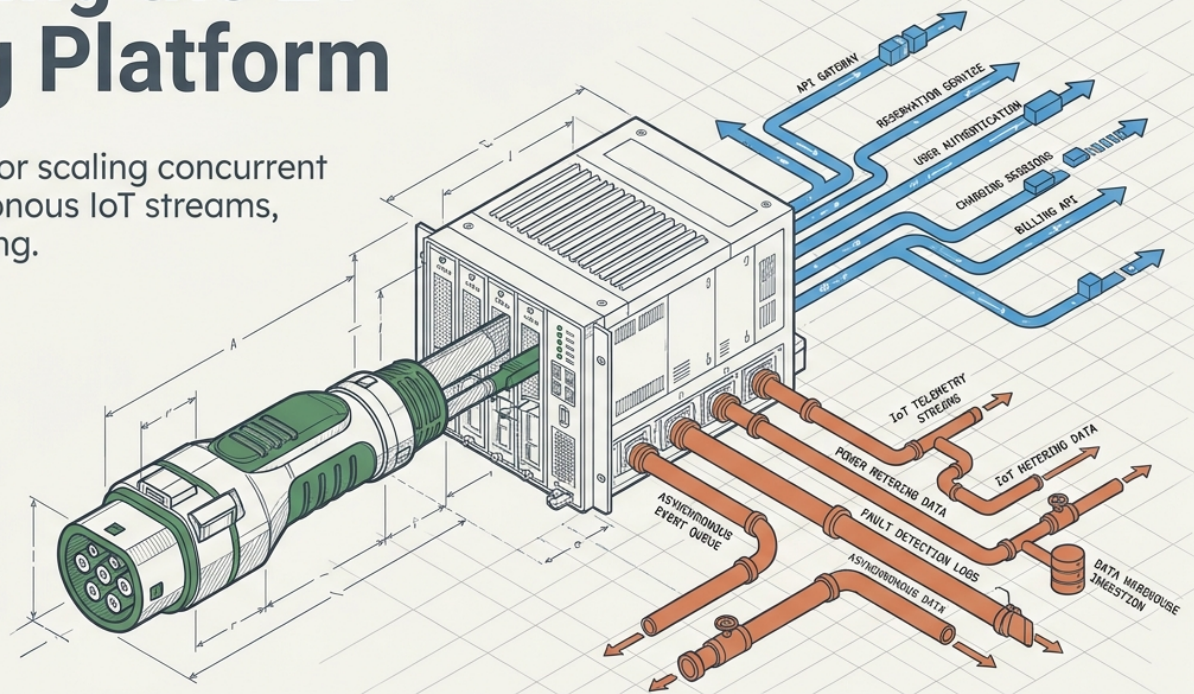


Architecting the EV Charging Platform

A definitive blueprint for scaling concurrent reservations, asynchronous IoT streams, and fault-tolerant billing.



SYSTEM_RUBRIC_V1.0 // NO FLUFF. JUST ARCHITECTURE.

The Core Architectural Tension

The Mandate

- Station discovery and real-time connector availability
- Concurrent reservations and session locking
- Charging session orchestration
- High-volume telemetry and device health
- Immutable pricing and idempotent payments
- Multi-tenant operator management

The Reality

- ⚠ **UNRELIABLE CHARGER CONNECTIVITY (FLAKY WEBSOCKETS)**
- ⚠ **DUPLICATE AND OUT-OF-ORDER NETWORK EVENTS**
- ⚠ **NETWORK PARTITIONS SEPARATING CHARGERS FROM THE BACKEND**
- ⚠ **CATASTROPHIC DEVICE AND HARDWARE FAILURES**

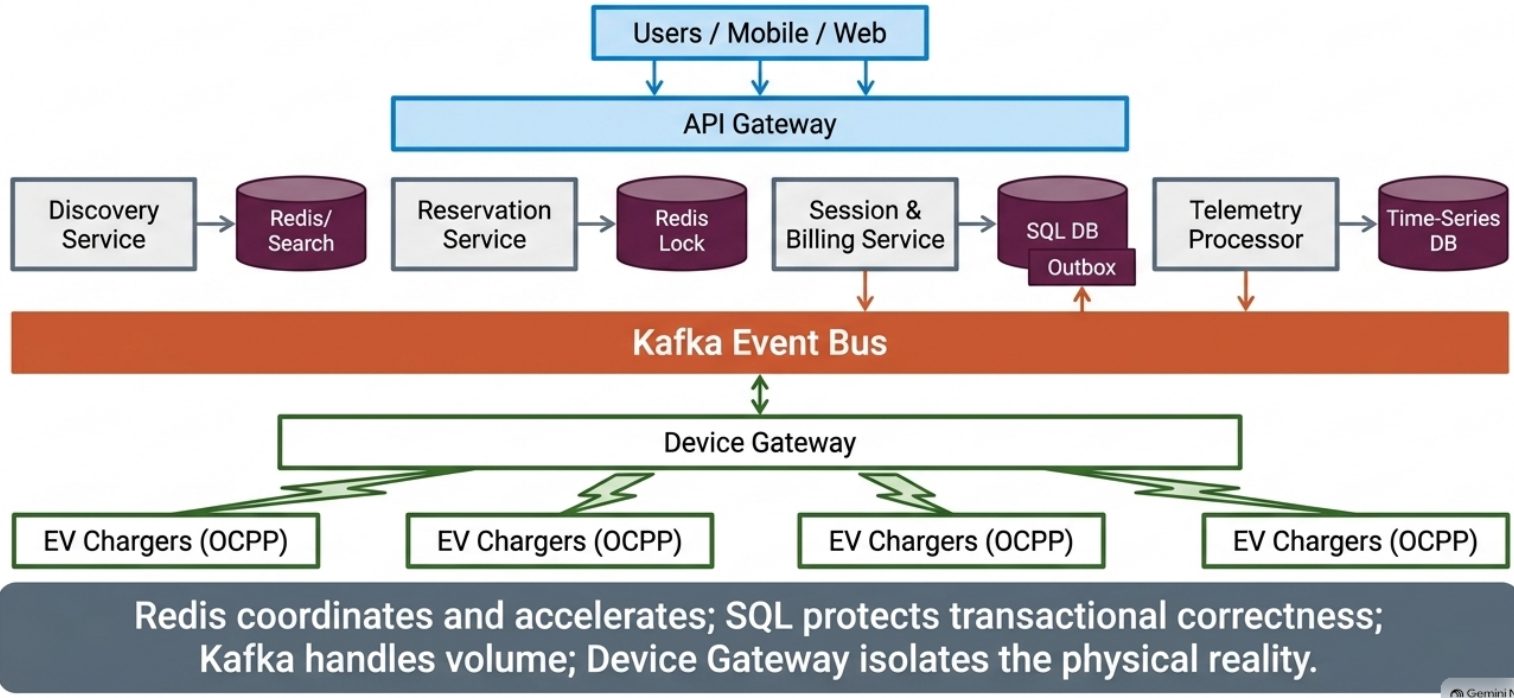
The objective is not just to route data, but to design a platform that guarantees 99.99% availability and strong transactional consistency over a highly chaotic physical IoT network.

Operating Parameters and Implied Architecture

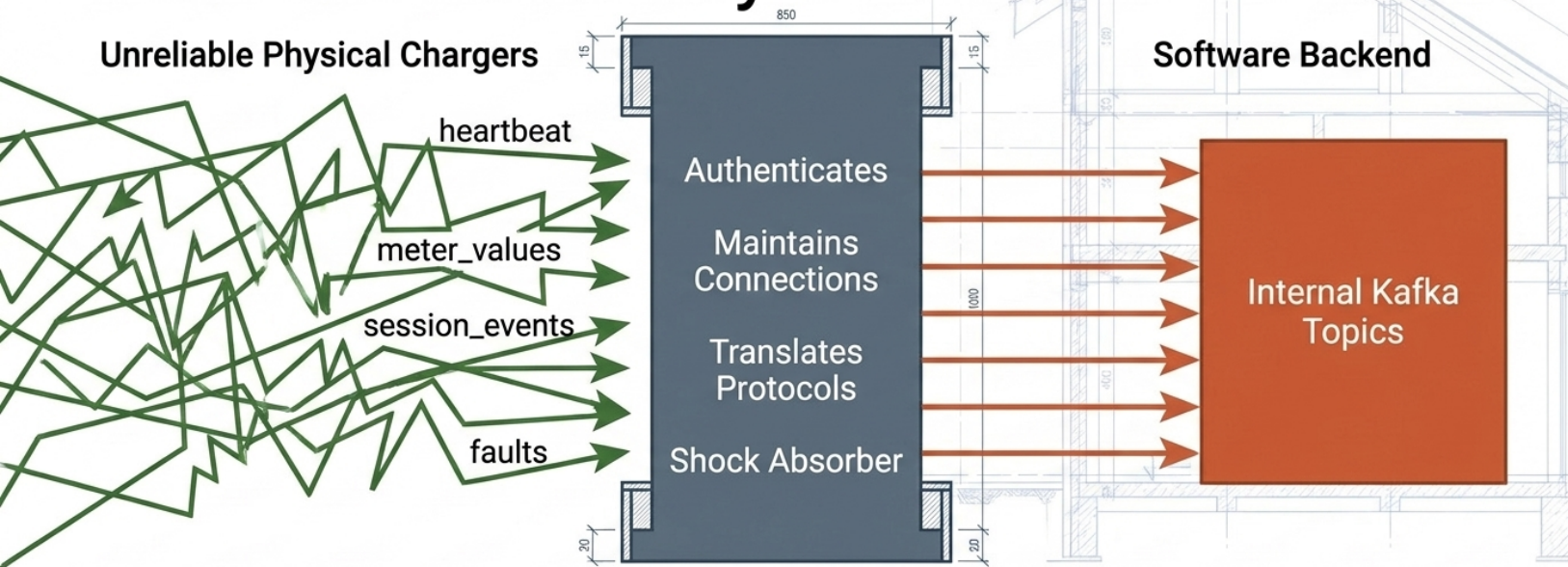
Scale Metric	Architectural Implication
100,000 Stations / 400,000 Connectors	Cache read-heavy station/status views; minimize geospatial query load on the core DB.
100,000 Concurrent sessions	Partition event streams by connector/session for independent, horizontal scaling.
50,000 Telemetry events/sec	Stream telemetry via Kafka; completely bypass the core transactional SQL database.
10,000 API requests/sec	Horizontally scale APIs/Gateways; keep reservation/billing paths strongly consistent.

Target Metrics: Availability: 99.99% | Discovery p95 Latency: < 500ms | Status Propagation: < 3 seconds

The High-Level System Blueprint



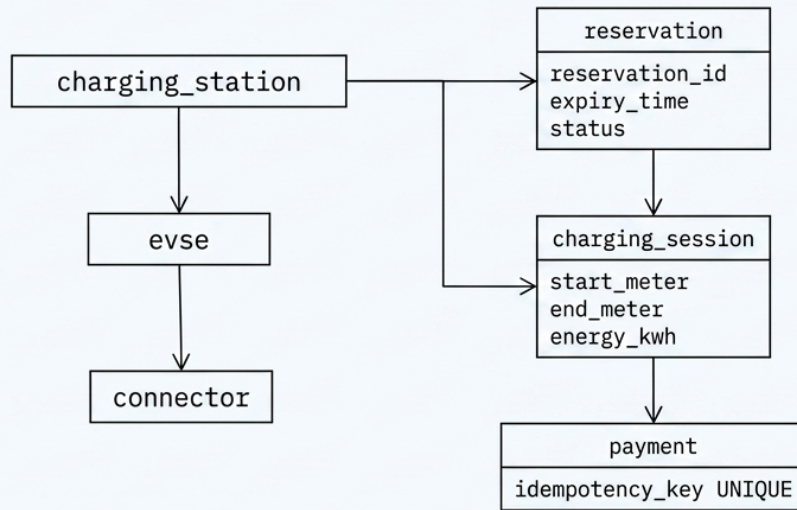
The Device Gateway Shield



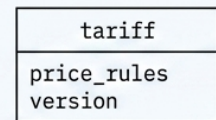
Never expose the core APIs directly to IoT devices. The gateway acts as an isolation barrier, translating messy WebSocket connections into predictable asynchronous events.

Entity-Relationship Domain Map

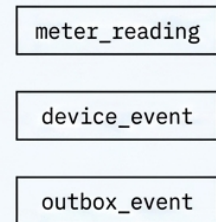
Core Transactional (SQL)



Configuration/Pricing (Cache)

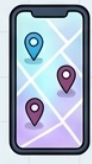


High-Volume Telemetry (Time-Series/Kafka)



Real-Time Availability & Geospatial Search

1. Search Phase



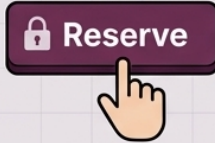
User App

Geospatial Search
Engine (Cache)

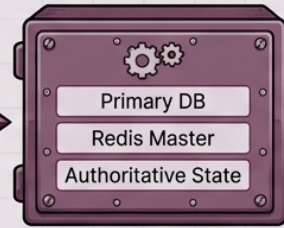


Returns slightly stale
candidate list

2. Intent Phase



Geospatial Search
Engine (Cache)



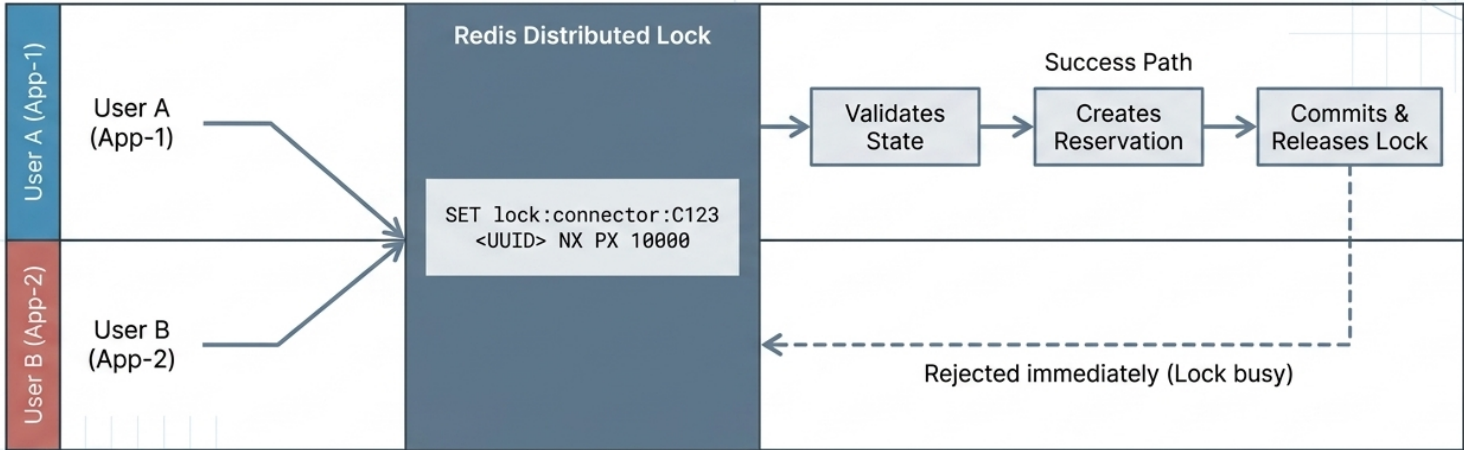
Core SQL / Redis Cluster



Revalidates authoritative
state before confirming

Accept eventual consistency for discovery. **Enforce strong consistency** for intent.
Search results guide the user, but the reservation path must always revalidate the ground truth.

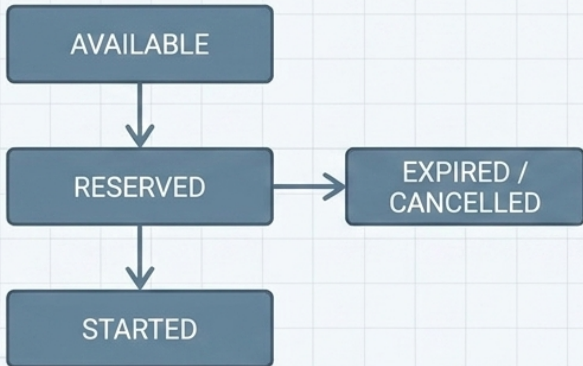
The Reservation Race Condition



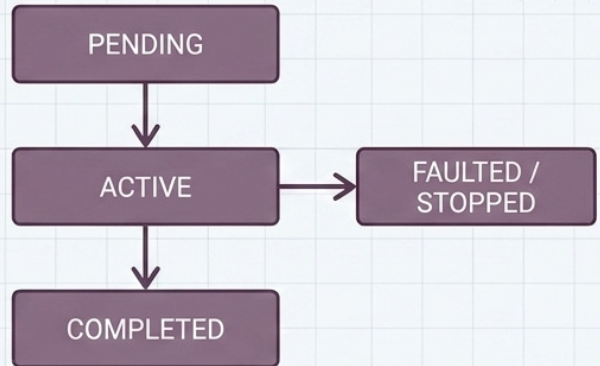
A JVM lock is insufficient for distributed applications. Use **Redis distributed locking**.
Crucially: keep the critical section short and NEVER hold the lock while waiting for payment gateways.

The State Matrix: Intent vs. Reality

Reservation State (The Intent)

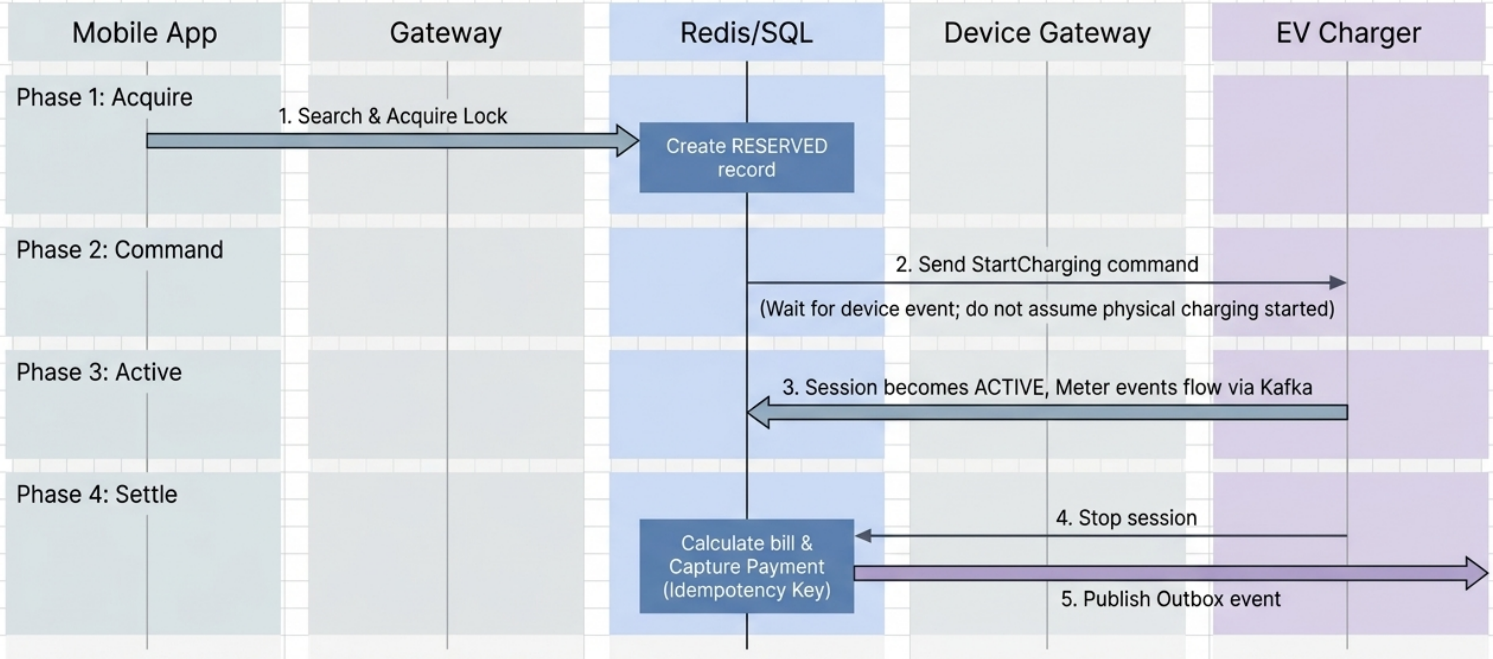


Charging Session State (The Reality)

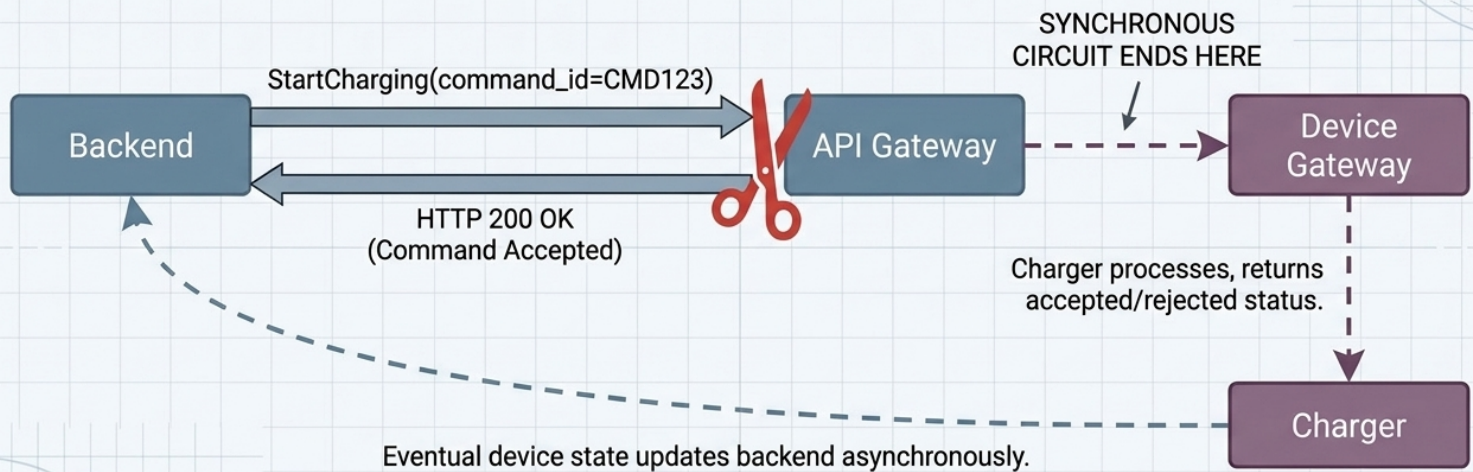


Keep reservation state distinct from actual charging-session state. Reserving a connector is a software guarantee; delivering energy is a physical reality. Conflating the two leads to system corruption when physical devices fail.

End-to-End Charging Orchestration Flow

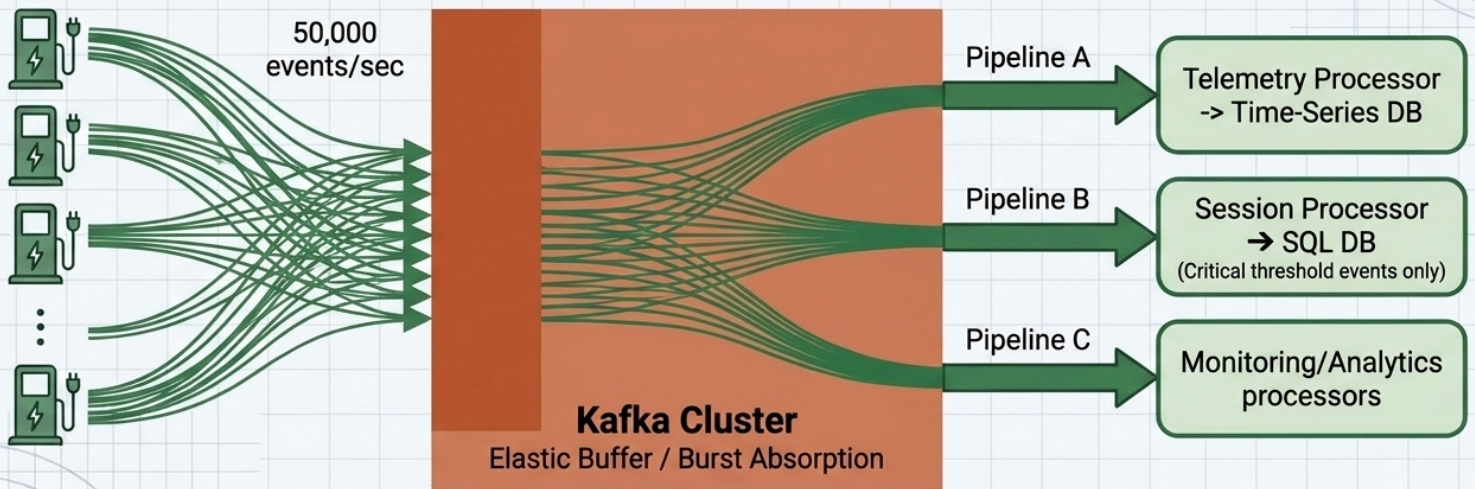


Asynchronous Command Execution



A successful API response means the command was queued for delivery, NOT that energy is flowing.
Track command state independently and reconcile against eventual device events.

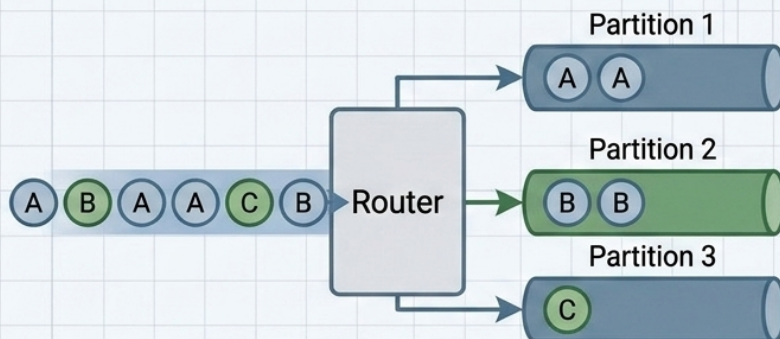
Handling the Telemetry Firehose



Never point raw telemetry at a transactional SQL database. Kafka buffers network bursts and allows independent scaling of downstream consumers.

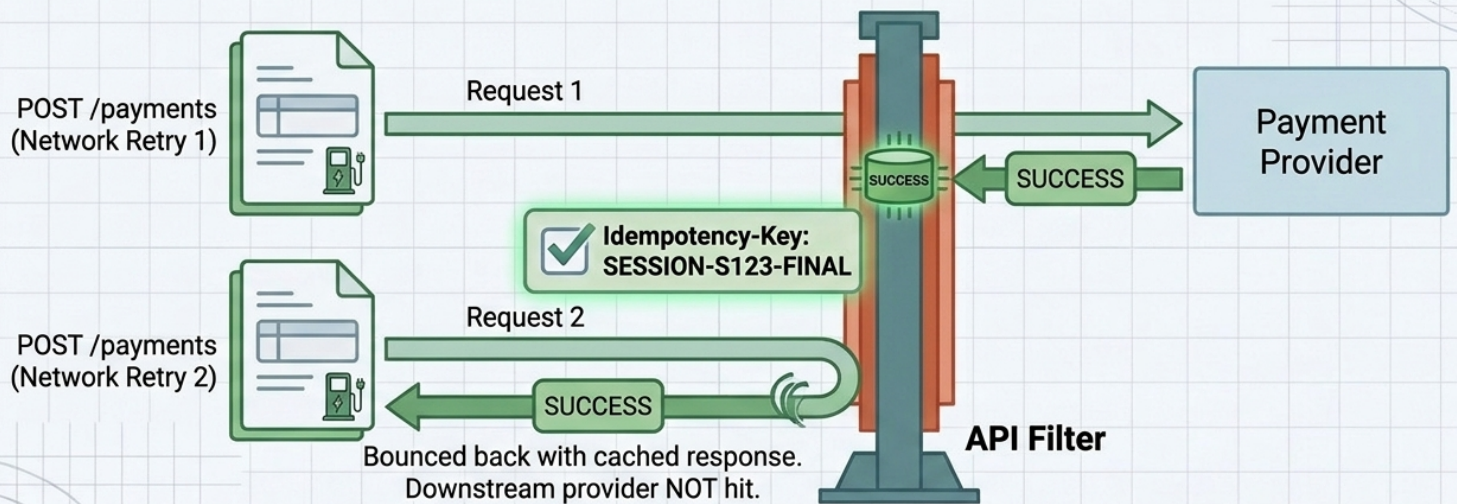
Kafka Partitioning & Ordering Guarantees

Strategy: Per-Connector Ordering	Key: connector_id	Use Case: Hardware status updates, fault sequencing
Strategy: Per-Session Ordering	Key: session_id	Use Case: Meter readings, billing calculations, lifecycle events



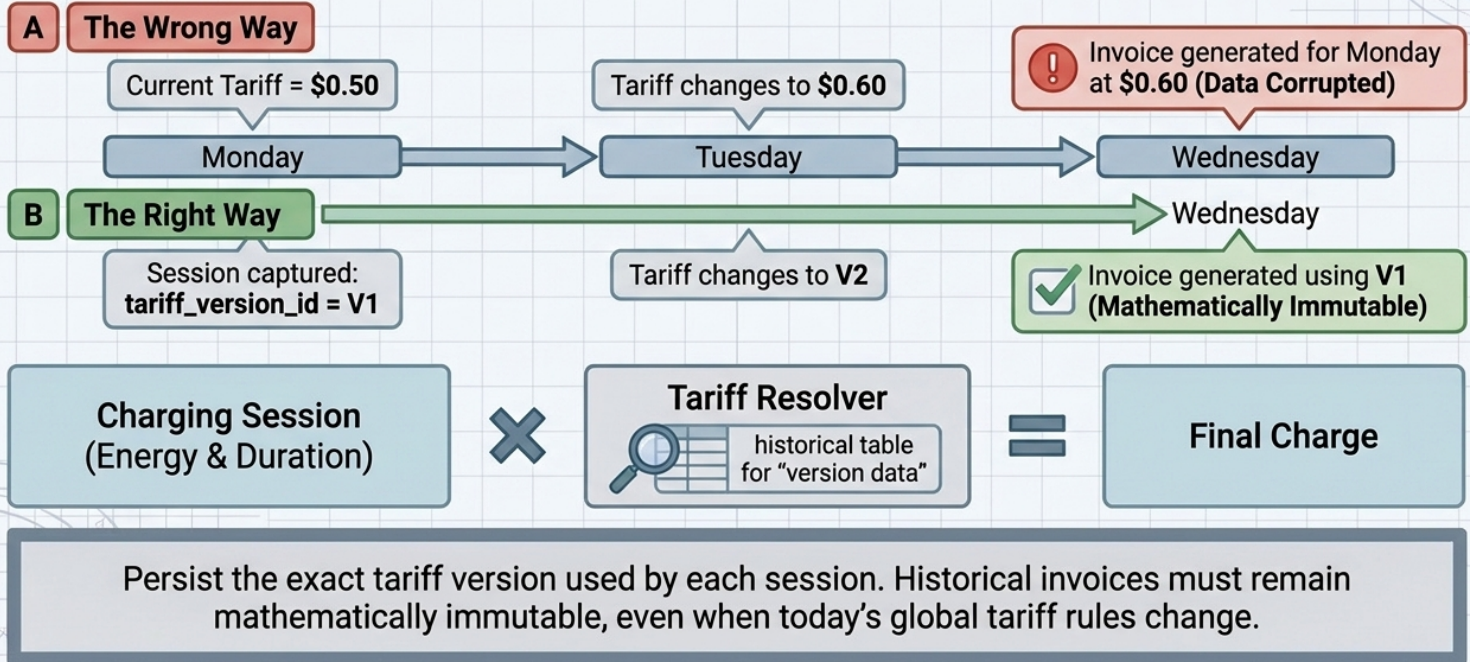
Identical keys map to the exact same partition queue, ensuring strict chronological processing.

The Idempotency Key Guard

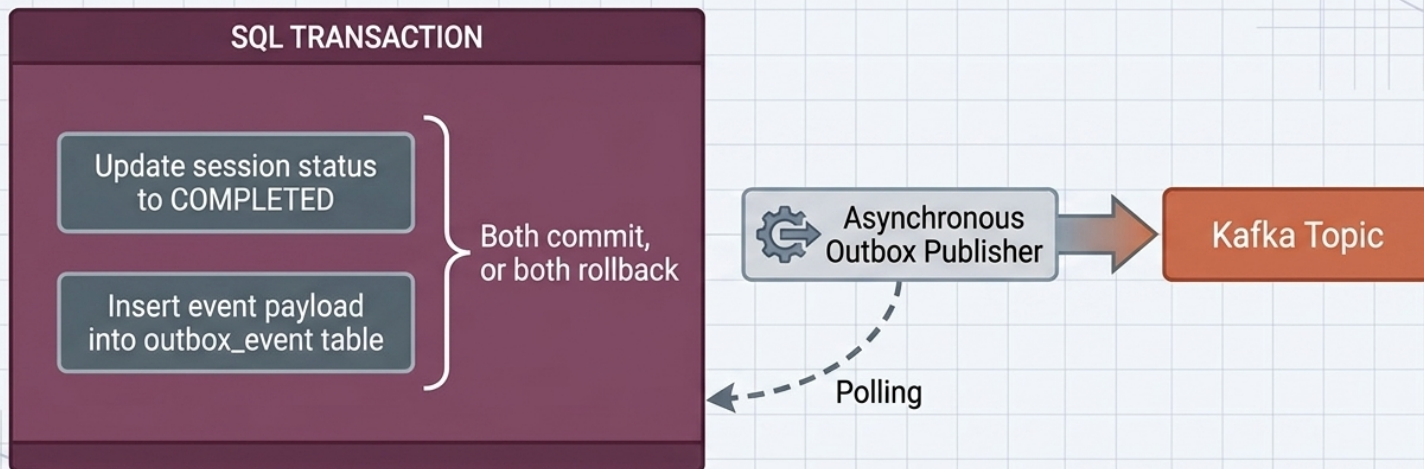


Consumers must tolerate duplicate delivery. Always rely on message IDs, unique constraints, and idempotent consumer patterns to prevent double-charging users.

Immutable Pricing & Tariff Integrity

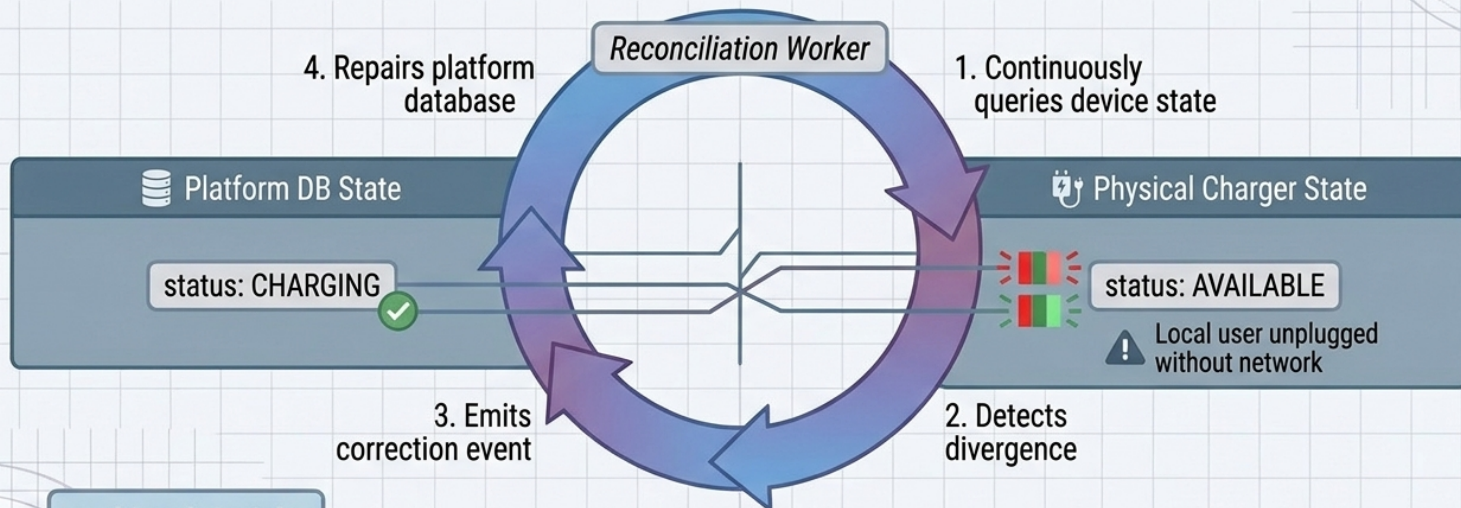


The Outbox Pattern Engine



Prevents data loss. Never write to the database and publish to Kafka in the same synchronous function. If Kafka goes down, the database thinks the session is complete, but downstream systems are never notified.

The Reconciliation Loop



Architect's Insight

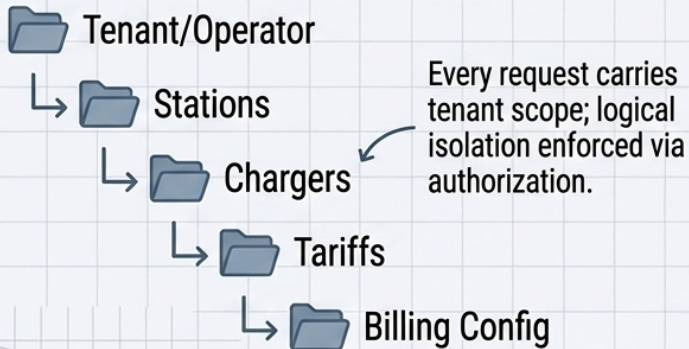
The fundamental truth of IoT architecture: The backend will eventually disagree with the physical device. A reconciliation loop is mandatory to repair the inevitable divergence between platform state and physical reality.

The Failure Response Matrix

Failure Scenario	Architectural Mechanism
Charger Offline	Heartbeat timeout -> degrade connector -> block reservations.
Network Partition	Persist state locally on device -> reconcile upon reconnect.
Duplicate Event	Deduplicate via sequence numbers and unique event IDs.
Stop Command Timeout	Run Reconciliation loop to verify physical state before billing.
Kafka Unavailable	Outbox table safely retains events until publishing succeeds.
Payment Timeout	Query payment provider using the exact same Idempotency Key.

Security & Multi-Tenancy Boundaries

Multi-Tenancy Tree



Security Arsenal



- ✓ Unique device identities
- ✓ TLS for all transport
- ✓ Least-privilege IAM
- ✓ Encrypted PII/payments
- ✓ Immutable audit logs

Remote operations (Start, Stop, Reboot) effectively control high-voltage physical equipment. These commands require the strictest authorization matrices and comprehensive auditability.

Observability & Distributed Tracing

1. Business Metrics

Revenue, reservation success/expiry rates, payment failure rates.

2. Platform Metrics

API p95 latency, Kafka consumer lag, DB connection pool utilization, Redis hit rates.

3. Device Metrics

Fleet heartbeat age, offline percentages, fault rates by firmware version, meter-event lag.



Core Principles of Distributed EV Architecture

- 1. Eventual Consistency Rules:** The physical world is eventually consistent; chargers will drift from platform state.
- 2. Separate Intent from Reality:** Accepting a command is not proof that physical charging started.
- 3. Idempotency Everywhere:** Device events, API retries, and payments must be safe to repeat infinitely.
- 4. Isolate High Volume:** Stream telemetry through Kafka to protect transactional databases.
- 5. Reconcile Constantly:** Automatically repair divergence between the database and the physical device.

The Architect Mindset: For every component, you must be able to explain why it exists, how it scales, how it fails, and the trade-offs you accepted.